

Física de Bolsillo — Manual Técnico

Documentación para desarrolladores · versión 1.0.0

Índice

Manual Técnico	1
1. Stack tecnológico	1
2. Estructura del proyecto	1
3. Capa de sensores: abstracción para pruebas	2
4. Procesamiento de audio: privacidad por diseño	2
5. FFT propia (sin librerías externas)	2
6. Base de datos: solo resúmenes	3
7. Visualización: Canvas de Compose	3
8. Inyección de dependencias manual	3
9. Pruebas automatizadas	3
10. Compilación	3
11. Limitaciones conocidas de esta entrega	4

Manual Técnico

1. Stack tecnológico

Componente	Tecnología
Lenguaje	Kotlin
UI	Jetpack Compose + Material 3
Arquitectura	MVVM (ViewModel + StateFlow)
Persistencia	Room (SQLite local)
Concurrencia	Coroutines + Flow
Sensores	android.hardware.SensorManager
Audio	android.media.AudioRecord (sin SpeechRecognizer, sin red)
Inyección de dependencias	Manual (AppContainer), sin Hilt/Dagger
JDK	17
minSdk	24 · targetSdk / compileSdk
Compilación	Gradle 8.9, AGP 8.5.x, KSP para Room

2. Estructura del proyecto

```
app/src/main/java/com/kidslab/pocketphysics/  
├── data/  
│   ├── local/          # Room: entidades, DAOs, AppDatabase, seed  
│   └── repository/      # Interfaces de repositorio (abstracción testable)
```

		sensors/	# SensorManagerRepository (real) + FakeSensorRepository (tests)
		audio/	# AudioRecordSoundRepository (real) + FakeSoundRepository (tests)
		domain/	
		model/	# Modelos de dominio puros (MotionSample, SoundAnalysisResult, ...)
		usecase/	# Lógica pura: FftUtil, AudioAnalyzer, MotionAnalysisUseCase, RotationAnalysis
		di/	# AppContainer: contenedor de dependencias manual
		ui/	
		theme/	# Colores y tipografía Material 3
		navigation/	# NavHost y rutas
		components/	# Gráfico Canvas reutilizable, bloques de pregunta/predicción/resultado
		screens/	# Un paquete por pantalla, con su ViewModel + Composable

3. Capa de sensores: abstracción para pruebas

El requisito de poder probar la lógica **sin hardware real** se resuelve con una interfaz `SensorRepository`:

```
interface SensorRepository {
    fun isSensorAvailable(type: SensorType): Boolean
    fun observeAccelerometer(): Flow<MotionSample>
    fun observeGyroscope(): Flow<RotationSample>
    fun observeLight(): Flow<LightSample>
}
```

`SensorManagerRepository` es la implementación real, apoyada en `SensorManager` y `callbackFlow`. `FakeSensorRepository` es una implementación en memoria usada en los tests, que emite una lista de muestras predefinida sin tocar `android.hardware`. Ningún `ViewModel` depende directamente de `SensorManager`.

El mismo patrón se repite para el micrófono con `SoundRepository` / `AudioRecordSoundRepository` / `FakeSoundRepository`.

4. Procesamiento de audio: privacidad por diseño

`AudioRecordSoundRepository` implementa un bucle de lectura sobre `AudioRecord` que:

1. Lee un bloque de `ShortArray` (PCM de 16 bits, 16 kHz, mono).
2. Llama a `AudioAnalyzer.analyze(...)`, una función **pura** que calcula amplitud promedio, amplitud máxima, frecuencia dominante (vía FFT propia) y una forma de onda simplificada de 64 puntos.
3. Emite el resultado por el `Flow` y **sobrescribe el mismo buffer** en la siguiente vuelta del bucle.

En ningún punto se escribe el PCM a disco, ni se acumula en una lista creciente, ni se envía a ningún servicio externo. No se usa `SpeechRecognizer` ni ninguna librería de reconocimiento de voz.

5. FFT propia (sin librerías externas)

Para la frecuencia dominante del sonido se implementó una FFT (Transformada Rápida de Fourier) de radix-2, iterativa, en Kotlin puro (`FftUtil.kt`), sin añadir dependencias externas de procesamiento de señales. Se aplica una ventana de Hann antes de transformar, para reducir fugas espectrales, y se ignora el bin de 0 Hz (offset de continua).

6. Base de datos: solo resúmenes

Ver [docs/BASE_DE_DATOS.md](#) para el esquema completo. El punto clave para el desarrollador: `ExperimentSessionDao.saveFullSession` guarda, en una única transacción, la sesión, la predicción, el `MeasurementSummary` (agregado) y el `ExperimentResult`. Nunca se persiste una lista de muestras de sensor.

7. Visualización: Canvas de Compose

`SimpleLineChart` y `ComparisonBar` (en `ui/components/SimpleLineChart.kt`) son gráficos dibujados a mano con Canvas de Compose. No se añadió ninguna librería de gráficos de terceros. El muestreo se limita a un historial acotado (por ejemplo, últimas 120 muestras) para no acumular memoria indefinidamente ni bloquear el hilo principal.

8. Inyección de dependencias manual

`AppContainer` (en `di/AppContainer.kt`) construye una única vez la base de datos y los repositorios, y `AppViewModelFactory` los inyecta en cada `ViewModel`. No se usa Hilt ni Dagger, siguiendo el mismo patrón que `HabitHero` y `Money Explorer`.

9. Pruebas automatizadas

Todas las pruebas corren en la JVM (con Robolectric cuando se necesita un Context de Android), **sin hardware real ni emulador**:

Archivo	Qué prueba
<code>MotionAnalysisUseCaseTest</code>	Resumen (mín/máx/promedio) y detección de movimiento con datos simulados.
<code>RotationAnalysisUseCaseTest</code>	Resumen de rotación y estimación de ángulo con velocidad angular constante.
<code>FftUtilTest</code>	Frecuencia dominante detectada correctamente en señales seno sintéticas.
<code>AudioAnalyzerTest</code>	Amplitud, forma de onda y frecuencia con PCM sintético (sonidos suaves/fuertes, graves/agudos).
<code>SensorAvailabilityTest</code>	Comportamiento cuando un sensor no existe en el dispositivo.
<code>AppDatabasePersistenceTest</code>	Persistencia real con Room en memoria: guardar y recuperar una sesión completa.
<code>ChallengeRepositoryTest</code>	Las insignias se otorgan una sola vez por desafío.
<code>MicrophonePermissionTest</code>	El flujo de audio falla de forma controlada si <code>RECORD_AUDIO</code> no fue concedido.

Ejecutar todos los tests:

```
./gradlew testDebugUnitTest
```

10. Compilación

```
./gradlew assembleDebug    # APK de depuración
./gradlew assembleRelease  # APK de release (sin firmar en esta versión)
```

11. Limitaciones conocidas de esta entrega

- El binario `gradle-wrapper.jar` no se incluyó en el repositorio (el entorno de generación no tenía acceso de red para descargarlo). Antes de compilar por primera vez, ejecuta `gradle wrapper` con una instalación local de Gradle, o dejar que GitHub Actions lo resuelva mediante el `distributionUrl` ya configurado en `gradle-wrapper.properties`.
- El APK de release no está firmado con una clave de producción; para publicar en Google Play sería necesario configurar firma y ofuscación.